CSED233 · 데이터구조 · 포스텍

정리본 7. Heaps

Priority Queue의 동기 · Heap의 두 성질 · Insert/DeleteMin · Heapify가 O(n)인 이유.

1. Priority Queue(우선순위 큐)

일반 큐는 먼저 온 것부터 처리한다(FIFO). 그런데 응급실에서 응급(3) > 중증(2) > 일반(1)처럼 우선순위가 다르면? 일반 큐는 [일반, 응급, 중증] 순으로 들어와도 일반부터 처리한다 — 우선순위를 무시한다.

Priority Queue(우선순위 큐): 들어온 순서가 아니라 우선순위 순으로 꺼내는 큐.

- `Insert(x)` : 우선순위를 유지하며 삽입.
- `DeleteMin()` : Min-Priority Queue(Min-PQ)에서 최소 키를 가진 원소를 삭제하면서 반환한다. 일반적으로 Min-Heap으로 구현한다.
- `DeleteMax()` : Max-Priority Queue(Max-PQ)에서 최대 키를 가진 원소를 삭제하면서 반환하는 별개 연산이다. 일반적으로 Max-Heap으로 구현하며, DeleteMin()과 혼용해서는 안 된다.

예: Insert(10, 5, 8, 3) → DeleteMin()을 반복하면 3, 5, 8, 10 순으로 자동 정렬되어 나온다.

응급실 예시처럼 숫자가 클수록 긴급하다고 정의하면 Max-PQ를 사용하며 DeleteMax()로 가장 긴급한 환자를 꺼낸다. 반대로 숫자가 작을수록 긴급하다고 정의한 경우에는 Min-PQ와 DeleteMin()을 사용한다.

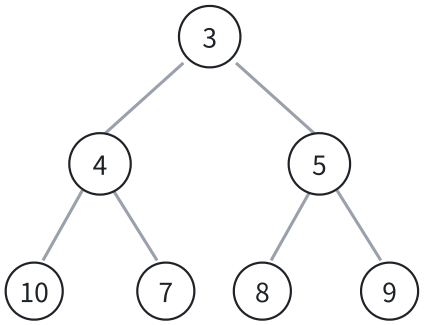
1.1 구현 방법에 따른 성능 — Heap이 "세 번째 정답"

구현	Insert	DeleteMin	병목
정렬 안 된 리스트	$O(1)$	$O(n)$ (최솟값 탐색)	DeleteMin
정렬된 리스트	$O(n)$	$O(1)$	Insert
Heap	$O(\log n)$	$O(\log n)$	**없음 — 둘 다 빠름**

2. Heap의 두 가지 성질

Heap의 조건

1. **Structural Property(구조 조건)**: Complete Binary Tree — 마지막 레벨 전까지 가득, 마지막은 왼쪽부터. 그래서 배열로 구현 가능.
2. **Heap Order Property(순서 조건)**: Min-Heap은 부모 ≤ 자식(루트 = 최소), Max-Heap은 부모 ≥ 자식(루트 = 최대).



3	4	5	10	7	8	9
[1]	[2]	[3]	[4]	[5]	[6]	[7]

Min-Heap 확인: 3≤4 ✓ 3≤5 ✓ 4≤10 ✓ 4≤7 ✓ 5≤8 ✓ 5≤9 ✓
배열 그대로: [3, 4, 5, 10, 7, 8, 9]

완전 이진 트리 모양이므로 배열과 1:1 대응

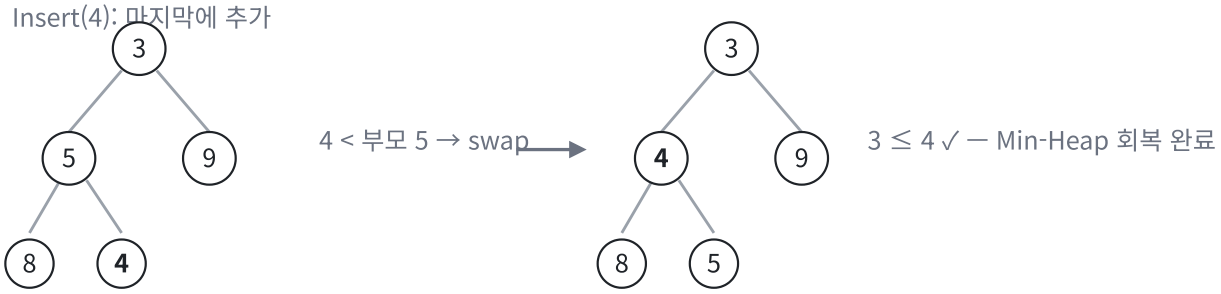
3. 기본 연산: Insert와 DeleteMin

3.1 Insert — Up-heap Bubbling, $O(\log n)$

① 새 원소를 **마지막 위치**(다음 빈 칸)에 넣는다 — Complete 형태 유지.

② 부모와 비교해 **부모보다 작으면 swap** — 위로 거슬러 올라간다.

③ 부모 ≤ 자식이 되면 멈춘다. 최악 = 트리 높이.



3.2 DeleteMin — Down-heap Bubbling, $O(\log n)$

① 루트(최솟값)를 꺼내고, **마지막 노드를 루트로** 옮긴다.

② 두 자식 중 **더 작은 쪽**과 비교해 자식이 더 작으면 swap — 아래로 내려간다.

③ 자식이 하나뿐이면 그 유일한 자식과 직접 비교해 자식이 더 작으면 swap한다. 완전이진트리에서 자식이 하나뿐이라면 그 자식은 반드시 왼쪽 자식이다. 자식이 없으면(리프) 즉시 멈춘다.

④ 부모 ≤ 자식이 되면 멈춘다.

예제 · DeleteMin

```
   3        8        4        4
  / \      / \      / \      / \
 4   9  →  4   9  →  8   9  →  5   9
/ \      /        /        /
5   8    5        5        8
(1) 루트 삭제   (2) 8 > 4 → swap   (3) 8 > 5 → swap
    마지막 8을 루트로
```

배열: [3,4,9,5,8] → 3 삭제·8을 루트로 [8,4,9,5] → swap(8,4) [4,8,9,5] → swap(8,5) [4,5,9,8]

> **Down-heap에서 비교 상대**
>
> 자식 둘 중 **아무하고나** 비교하면 안 된다. 반드시 **더 작은 자식**과 비교해야 swap 후에도 Min-Heap 성질(부모 ≤ 두 자식 모두)이 보장된다.

4. Heap Sort(힙 정렬)

① 배열을 Max-Heap으로 만든다 → $O(n)$ (아래 Heapify).

② 현재 힙 크기를 m이라 할 때, 루트(최댓값)와 힙의 마지막 원소를 교환해 최댓값을 뒤쪽에 확정한다.

③ 힙 크기를 m−1로 줄인다 — 확정된 원소는 이미 정렬된 영역이라 이후 힙 연산에 참여하지 않는다.

④ 루트로 올라온 원소에 Down-heap을 수행해 Max-Heap 속성을 복구한다.

⑤ 힙 크기가 1이 될 때까지 교환·감소·복구를 반복하면 배열 전체가 오름차순으로 정렬된다.

총 $O(n \log n)$, 공간은 $O(1)$ (in-place, 배열 내부 정렬·반복식 Down-heap 기준).

5. Heapify — 힙을 O(n)에 만드는 법

방법	과정	시간
① 순차 삽입	빈 힙에 n번 Insert	$O(n \log n)$ (최악)
② **Heapify**	원소를 전부 배열에 넣은 뒤, **마지막 비단말 노드($\lfloor n/2 \rfloor$)부터 루트까지 역순**으로 Down-heap	$O(n)$

왜 끝에서부터인가: Down-heap은 자식 서브트리가 이미 힙이어야 부모를 끌어내릴 수 있다. 리프는 그 자체로 힙이므로, 가장 작은 서브트리부터 Bottom-up으로 합쳐 올라간다.

예제 · Min-Heap → Max-Heap 변환 (n=10)

초기: [4, 5, 9, 6, 8, 9, 10, 10, 18, 9] (1-indexed)
인덱스 5(=⌊10/2⌋) → 4 → 3 → 2 → 1 순서로 Down-heap

결과: [18, 10, 10, 6, 9, 9, 9, 5, 4, 8]



```
 6  9 9  9
/\ /
5 4 8
```

왜 Heapify는 O(n)인가

- 노드의 높이를 그 노드에서 가장 깊은 자손 리프까지의 간선 수로 두자. 인덱스 i인 노드의 높이가 j 이상이려면 $2^j \cdot i \le n$이어야 하므로, 높이가 정확히 j인 노드 수 $N_j = \lfloor n/2^j \rfloor - \lfloor n/2^{j+1} \rfloor$ 이고, 따라서 $N_j \le n/2^j$ 이다.
- 높이가 j인 노드의 Down-heap은 최대 j레벨을 내려가므로, 레벨당 비용을 상수 c로 두면 전체 작업량 $T(n) \le c \cdot (\text{j=0..h 구간의 } \Sigma\ j \cdot N_j) \le cn \cdot (\text{j=0..}\infty \text{ 구간의 } \Sigma\ j/2^j) = cn \cdot 2 = 2cn = O(n)$이다. 여기서 $\Sigma\ j/2^j = 2$는 급수 공식 $\Sigma\ j \cdot x^j = x/(1-x)^2$ ($|x| < 1$)에 $x = 1/2$를 대입한 값이다.
- 트리 아래쪽(높이 작은 쪽)에 노드가 몰려 있어 대부분의 노드는 조금만 이동한다.

$$\sum_{k=0}^{h} 2^k (h-k) \;\le\; n \sum_{j=0}^{h} \frac{j}{2^j} \;\le\; 2n \;=\; O(n)$$

한 장의 그림으로 말하면: Insert n번은 "모든 노드가 루트까지 오를 수 있는" 비용을 치르지만, Heapify는 절반은 0번, 1/4은 1번, 1/8은 2번만 이동한다.

6. Quiz

Q1. Heap이 정렬된 리스트보다 나은 이유는?

힌트

섹션 1.1 표: 정렬된 리스트는 한쪽 연산이 O(n)이다.

정답

개념은 힌트 참고.

정답: 정렬된 리스트는 Insert O(n) + DeleteMin O(1)이라 Insert가 병목. Heap은 둘 다 O(log n)이라 전체 효율이 낫다.

Q2. Min-Heap에서 두 번째로 작은 값은 어디 있나?

힌트

Heap order는 "부모 ≤ 자식"만 보장한다. 형제끼리의 순서는?

정답

개념은 힌트 참고.

정답: 서로 다른 키를 갖는 Min-Heap에서 원소 수가 n ≥ 3이면, 두 번째로 작은 키는 반드시 루트의 두 자식 중 키가 더 작은 자식에 위치한다. 단, n = 2이면 루트에는 왼쪽 자식 하나만 있으므로 두 번째로 작은 키는 그 유일한 자식에 존재하고, n = 1이면 두 번째로 작은 원소는 존재하지 않는다. 어느 쪽 자식인지는 사전에 정해져 있지 않다 (Heap은 부분 정렬).

Q3. Heap이 배열로 구현되는 이유는?

힌트

정리본 5의 Complete Binary Tree 배열 구현과 같은 원리.

정답

개념은 힌트 참고.

정답: Complete Binary Tree라 빈 공간 없이 배열에 차곡차곡 들어가고, 인덱스 계산으로 부모-자식을 바로 찾을 수 있어서. 1-indexed 배열에서 인덱스 i인 노드의 부모는 ⌊i/2⌋ (i ≥ 2인 경우), 왼쪽 자식은 2i, 오른쪽 자식은 2i + 1이며, 계산된 자식 인덱스가 n보다 크면 그 자식은 존재하지 않는다.

Q4. Up-heap과 Down-heap의 차이는?

힌트

어느 연산 후에, 어느 방향으로 움직이는가.

정답

개념은 힌트 참고.

정답: Up-heap은 Insert 후 루트 방향으로 거슬러 올라가며 회복, Down-heap은 Delete 후 리프 방향으로 내려가며 회복.

Q5. Heapify에서 루트가 아니라 마지막 비단말 노드부터 시작하는 이유는?

힌트

Down-heap의 전제 조건: 자식 서브트리가 이미 힙이어야 한다. 리프는 그 자체로 힙.

정답

개념은 힌트 참고.

① Down-heap은 자식들이 이미 올바른 힙일 때만 부모를 올바른 위치로 끌어내릴 수 있다.

② 리프 노드는 자식이 없으므로 그 자체로 힙이다.

③ 따라서 리프 바로 위의 마지막 비단말 노드(⌊n/2⌋)부터 시작해 루트까지 거슬러 올라가며 힙을 점진적으로 병합한다.

정답: 자식 서브트리가 먼저 힙이어야 Down-heap이 성립하므로, 가장 작은 서브트리부터 Bottom-up으로 처리해야 하기 때문.